

NBA ANALYTICS

Pipeline de Engenharia de Dados — Guia Completo

dbt Core · PostgreSQL · Python · Selenium · Dagster · Docker · GitHub Actions

■ dbt Core — conceitos e camadas	■ Docker Compose — ambiente reproduzível
■ Cada arquivo e pasta explicado	■■ Dagster — orquestração e assets
■ Regras de negócio implementadas	■ GitHub Actions — CI/CD automático
■ Testes de qualidade de dados	■ Guia completo de entrevista Pleno/Sênior

PARTE 1 — Visão Geral do Projeto

1.1 O que é este projeto

Pipeline de dados end-to-end para análise de estatísticas da NBA. Extrai dados do Basketball Reference via Selenium, armazena em PostgreSQL, transforma com dbt Core seguindo arquitetura em três camadas, orquestra com Dagster e valida com GitHub Actions CI.

1.2 Stack tecnológica completa

Camada	Ferramenta	Versão	Função
Extração	Python + Selenium	4.31	Web scraping do Basketball Reference
Armazenamento	PostgreSQL	17	Data warehouse local
Ambiente	Docker Compose	v2	Banco reproduzível em qualquer máquina
Transformação	dbt Core	1.9.10	Modelos SQL em 3 camadas
Orquestração	Dagster	latest	Agendamento, assets, observabilidade
Qualidade	dbt tests (YAML)	—	26 testes declarativos automatizados
CI/CD	GitHub Actions	—	Compile + seed + run + test em cada PR
Docs	dbt docs + GH Pages	—	Documentação publicada automaticamente

1.3 Fluxo completo do pipeline

Basketball Reference

```
■  
■ Selenium (headless Chromium) – src/scraping/  
▼  
seeds/*.csv (camada raw)  
■  
■ dbt seed → analytics_raw.*  
▼  
models/staging/bbr/ → analytics_staging.* (views – limpeza)  
■  
■ dbt run  
▼  
models/intermediate/ → analytics_intermediate.* (views – regras de negócio)  
■  
▼  
models/marts/ → analytics_marts.* (tables – dim_* e fct_*)  
■  
■ dbt test → 26 testes de qualidade  
▼  
Dagster UI → observabilidade, agendamento, histórico de runs  
GitHub Actions → CI automático a cada PR
```

PARTE 2 — dbt Core: Conceitos e Camadas

2.1 O que é dbt e por que ele existe

dbt (data build tool) é uma ferramenta que permite escrever transformações de dados em SQL puro, com dependências automáticas entre modelos, testes de qualidade e documentação gerada do próprio código. É o padrão de mercado para a camada T do ELT.

■ Sem dbt	■ Com dbt
SQL em arquivos numerados sem ordem definida	Ordem automática via grafo de dependências (DAG)
CREATE TABLE IF NOT EXISTS na mão	Materialização automática (view, table, incremental)
Zero testes de dados	Testes declarativos em YAML (unique, not_null...)
Documentação desatualizada em wiki	Docs gerados do código, sempre sincronizados
'Quem usa essa tabela?' — ninguém sabe	Lineage graph mostra impacto de cada mudança

2.2 A arquitetura de 3 camadas

Camada	Pasta	Materialização	Responsabilidade única
Staging	models/staging/bbr/	VIEW	Limpeza: tipos, nomes, filtros. Sem lógica de negócio.
Intermediate	models/intermediate/	VIEW	Regras de negócio reutilizáveis (ex: dedup jogadores).
Marts	models/marts/	TABLE	Modelo dimensional final para analistas e dashboards.

■ **Princípio fundamental:** se o Basketball Reference mudar a estrutura de uma página, APENAS a staging precisa mudar. Intermediate e marts confiam nos contratos da staging (nomes e tipos estáveis). Uma mudança na fonte não quebra 5 modelos ao mesmo tempo.

2.3 Convenção de nomenclatura dos modelos

Padrão	Exemplo	Camada
stg_[fonte]__[entidade]	stg_bbr__player_stats	Staging
int_[domínio]__[propósito]	int_players__deduped	Intermediate
dim_[entidade]	dim_player	Marts — dimensão
fct_[processo]	fct_player_season_stats	Marts — fato

O duplo underscore (__) separa fonte de entidade — convenção oficial dbt. Quando há múltiplas fontes (BBR, Sportradar, ESPN), fica imediatamente claro de onde cada modelo lê seus dados.

2.4 O problema do jogador trocado — regra de negócio real

Quando um jogador muda de time, o BBR insere múltiplas linhas: uma por time e uma agregada (2TM, 3TM...). Sem tratamento, um jogador de 3 times aparece 4 vezes — somando jogos daria 220 numa temporada de 82.

```
-- int_players__deduped.sql
traded_players as (
  select distinct player_name from players
  where team_abbr = 'TOT'          -- formato legado BBR
     or team_abbr ~ '^\\d+TM$'     -- formato atual: 2TM, 3TM...
),
deduped as (
  select p.* from players p
  left join traded_players t using (player_name)
  where t.player_name is null      -- não foi trocado: mantém
     or p.team_abbr ~ '^\\d+TM$'  -- foi trocado: só o agregado
)
```

■ **Descoberto pelos testes:** após a correção, os testes de unicidade confirmaram que `int_players__deduped` e `int_player_stats__season_totals` têm exatamente uma linha por jogador. Sem testes, o bug teria passado silenciosamente.

2.5 Modelo dimensional (Star Schema)

Os marts implementam o Star Schema de Ralph Kimball: uma tabela fato central com métricas numéricas, conectada a tabelas dimensão com atributos descritivos.

Tabela	Tipo	Grain / Conteúdo	Surrogate Key
dim_player	dim	1 linha por jogador ativo na temporada	MD5(player_name)
dim_team	dim	1 linha por franquia ativa (30 times)	MD5(abbreviation)
fct_player_season_stats	fct	1 linha por jogador por temporada (grain)	MD5(player+season)

■ **Surrogate keys com MD5:** chaves artificiais geradas pela macro `generate_surrogate_key()`. Mais estáveis que nomes naturais (que podem mudar de grafia) e compatíveis com joins entre sistemas que não compartilham sequências de ID.

PARTE 3 — Docker Compose: Ambiente Reprodutível

3.1 Por que Docker é fundamental para portfólio

Sem Docker, o projeto funciona apenas na sua máquina, com o PostgreSQL que você instalou manualmente, na porta que você configurou. Um recrutador que quiser rodar o projeto precisa de um tutorial de 20 passos antes de ver qualquer dado.

Com Docker Compose, qualquer pessoa executa dois comandos e tem tudo funcionando:

```
docker compose up -d    # sobe o PostgreSQL em background
source .venv/bin/activate
dbt seed --profiles-dir .dbt && dbt run --profiles-dir .dbt && dbt test --profiles-dir .dbt
```

3.2 docker-compose.yml — o que cada parte faz

```
services:
  postgres:
    image: postgres:17-alpine          # imagem oficial, versão Alpine (menor)
    container_name: nba_postgres
    restart: unless-stopped           # reinicia automaticamente se o Docker reiniciar
    environment:
      POSTGRES_DB:      ${DBT_DBNAME:-nba}      # lê do .env ou usa 'nba'
      POSTGRES_USER:    ${DBT_USER:-postgres}
      POSTGRES_PASSWORD: ${DBT_PASSWORD:-postgres}
    ports:
      - '${DBT_PORT:-5432}:5432'             # expõe para o host
    volumes:
      - postgres_data:/var/lib/postgresql/data # dados persistem entre restarts
    healthcheck:
      test: pg_isready -U postgres          # GitHub Actions espera este check passar
```

3.3 A variável \${VAR:-default} — Docker Compose + dbt integrados

As mesmas variáveis de ambiente do .env alimentam tanto o Docker Compose (que cria o banco com aquelas credenciais) quanto o dbt (que conecta usando env_var()). Não há duplicação de configuração — um único .env controla tudo.

■ **Integração perfeita:** Docker cria o banco com DBT_PASSWORD=postgres, dbt lê DBT_PASSWORD=postgres para conectar. Mudar a senha exige alterar apenas o .env.

■ **Pergunta de entrevista:** Como você garante que o ambiente de desenvolvimento é igual ao de produção?

Com Docker Compose, o PostgreSQL roda na mesma versão (17-alpine) em qualquer ambiente — dev local, CI no GitHub Actions e produção. As credenciais vêm de variáveis de ambiente (.env local, secrets no GitHub, variáveis de servidor em prod). O único diferencial é que em produção o volume de dados seria maior e usaria um servidor PostgreSQL gerenciado (RDS, CloudSQL) em vez de container, mas a interface é idêntica.

PARTE 4 — Dagster: Orquestração Moderna

4.1 Dagster vs Airflow — a diferença de paradigma

Airflow foi criado em 2014 e pensa em **tasks**: você define o que executar e em que ordem. Dagster (2018) pensa em **assets**: você define quais *dados devem existir* e ele descobre o que rodar para produzi-los.

	Airflow	Dagster
Conceito central	Task / DAG	Asset (dado que deve existir)
Integração com dbt	BashOperator manual	dbt_assets — lê o projeto dbt nativamente
Observabilidade	Logs de task	Histórico por asset, materializações, metadados
UI	Grafo de tasks (técnico)	Grafo de assets (intuitivo para negócio)
Re-execução parcial	Re-roda tasks manualmente	Materializa só os assets desatualizados
Curva de aprendizado	Alta (conceitos de XCom, DAG, hook)	Média (assets são intuitivos)

4.2 Como o Dagster lê o projeto dbt automaticamente

O decorador `@dbt_assets` lê o `manifest.json` gerado por 'dbt compile' e cria um asset Dagster para cada modelo SQL automaticamente. O grafo de dependências do dbt (via `ref()`) é preservado no grafo de assets do Dagster.

```
# orchestration/assets.py
@dbt_assets(
    manifest=dbt_project.manifest_path, # lê models/staging, intermediate, marts
    project=dbt_project,
    deps=[scrape_players, scrape_stats, # assets de scraping alimentam os seeds
          scrape_teams, scrape_contracts],
)
def nba_dbt_assets(context, dbt: DbtCliResource):
    yield from dbt.cli(['build'], context=context).stream()
    # 'build' = seed + run + test em um único comando
```

4.3 Estrutura dos assets de scraping

```
@asset(
    group_name='scraping',
    description='Extrai roster de jogadores do BBR → seeds/players.csv',
    kinds={'python', 'selenium'},
)
def scrape_players(context: AssetExecutionContext) -> None:
    context.log.info('Iniciando scraping...')
    _run_scraper('players.py')

# O Dagster exibe cada asset na UI com:
# - Status (materializado/falhou/nunca rodou)
# - Última materialização (quando rodou pela última vez)
# - Logs completos de execução
# - Metadados (linhas produzidas, tempo de execução)
```

4.4 Schedule — execução automática semanal

```
# orchestration/definitions.py
nba_weekly_schedule = ScheduleDefinition(
    job=nba_pipeline_job,
    cron_schedule='0 6 * * 1',    # toda segunda-feira às 06:00
    name='nba_weekly_monday',
)

# Iniciar o Dagster:
dbt compile --profiles-dir .dbt    # gera manifest.json (obrigatório)
dagster dev -f orchestration/definitions.py
# Acesse: http://localhost:3000
```

■ Pergunta de entrevista: Por que você escolheu Dagster em vez de Airflow?

Airflow é o padrão histórico — toda empresa legada tem. Mas Dagster oferece duas vantagens concretas para este projeto: (1) integração nativa com dbt via `@dbt_assets`, que importa todos os modelos SQL automaticamente sem escrever task por task; (2) o conceito de asset é mais próximo do que o negócio entende — 'dim_player está atualizado?' é mais intuitivo do que 'a task dbt_run do DAG nba_pipeline passou?'. Em projetos novos sem legacy, Dagster é a escolha que times modernos fazem.

PARTE 5 — GitHub Actions: CI/CD Automático

5.1 O que o CI faz e por que importa

CI (Continuous Integration) garante que toda mudança nos modelos SQL é validada automaticamente antes de entrar na branch principal. Sem CI, um modelo quebrado só é descoberto quando alguém roda o pipeline manualmente — às vezes dias depois.

Evento	Workflow disparado	O que roda
Pull Request	dbt CI (ci.yml)	compile → seed → run → test — valida a mudança
Push em master	dbt CI + Docs	CI completo + gera e publica a documentação
Dispatch manual	Docs (docs.yml)	Regenera e publica a documentação sob demanda

5.2 Como o GitHub Actions sobe o PostgreSQL automaticamente

```
# .github/workflows/ci.yml
services:
  postgres:
    image: postgres:17-alpine
    env:
      POSTGRES_DB: nba
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    options: >-
      --health-cmd 'pg_isready -U postgres'
      --health-interval 10s
# O GitHub Actions sobe o container antes dos steps e o derruba depois.
# Nenhuma configuração manual — o banco aparece como 'localhost:5432'.
```

5.3 dbt docs publicado no GitHub Pages (docs.yml)

O workflow docs.yml roda o pipeline completo, executa 'dbt docs generate' e publica os arquivos gerados (index.html, manifest.json, catalog.json) no GitHub Pages. Resultado: a documentação do projeto fica disponível em uma URL pública, sempre atualizada com o último código da branch principal.

■ **Impacto no portfólio:** colocar a URL do dbt docs no README e no LinkedIn permite que qualquer recrutador ou engenheiro sênior explore o lineage graph e a documentação sem precisar instalar nada. É a demonstração mais direta de que você sabe construir pipelines documentados e testados.

■ **Pergunta de entrevista: O que acontece se um modelo SQL quebrar em um Pull Request?**

O workflow ci.yml é disparado automaticamente. Ele roda dbt compile (valida sintaxe), dbt seed (carrega CSVs), dbt run (executa os modelos) e dbt test (roda os 26 testes). Se qualquer step falhar, o PR fica bloqueado — não pode ser mergeado até a correção. O erro aparece diretamente na interface do GitHub PR, com o log completo. Ninguém precisa rodar nada manualmente para descobrir que algo quebrou.

PARTE 6 — Estrutura Completa de Arquivos

6.1 Mapa do projeto

```
NBA Analytics/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml          ← CI: compile+seed+run+test em cada PR
│   │   └── docs.yml        ← publica dbt docs no GitHub Pages
│   └── .dbt/
│       ├── profiles.yml    ← credenciais via env_var() (gitignored)
│       └── models/
│           ├── staging/bbr/ ← stg_bbr_*.sql (4 modelos)
│           ├── intermediate/ ← int_*_*.sql (2 modelos de dedup)
│           ├── marts/
│           │   ├── dimensions/ ← dim_player.sql, dim_team.sql
│           │   └── facts/      ← fct_player_season_stats.sql
│           ├── seeds/        ← CSVs scraped + team_info.csv estático
│           ├── macros/      ← generate_surrogate_key.sql
│           └── orchestration/
│               ├── assets.py ← assets Dagster (scraping + dbt)
│               ├── definitions.py ← schedule, jobs, resources
│               └── src/scraping/
│                   ├── common/ ← browser.py, parsing.py
│                   ├── players.py, stats.py, teams.py, contracts.py
│                   └── run_all.py ← orquestrador local (sem Dagster)
│               ├── diagrama/ ← PDFs de documentação
│               ├── docker-compose.yml ← PostgreSQL em container
│               ├── dbt_project.yml ← configuração central do dbt
│               ├── requirements.txt ← todas as dependências Python
│               ├── .env.example ← template de variáveis de ambiente
│               └── profiles.yml.example ← template do profiles.yml
```

6.2 Schemas no PostgreSQL

Schema	Criado por	Tipo	Consumido por
analytics_raw	dbt seed	TABLE	Modelos staging (via ref())
analytics_staging	dbt run	VIEW	Modelos intermediate
analytics_intermediate	dbt run	VIEW	Modelos marts
analytics_marts	dbt run	TABLE	Analistas, dashboards, Dagster UI

PARTE 7 — Qualidade de Dados

7.1 Os 26 testes declarativos

Todos os testes são declarados em YAML — o dbt gera o SQL de validação automaticamente. Rodam localmente (dbt test) e no CI (GitHub Actions) a cada PR.

Tipo de teste	Quantidade	Onde está declarado	O que valida
unique	8	marts + intermediate YAML	Chaves primárias sem duplicatas
not_null	12	todas as camadas YAML	Colunas obrigatórias preenchidas
accepted_values	4	seeds + marts YAML	conference = East ou West
relationships	2	marts YAML	FK de fct existe em dim

7.2 Como rodar os testes

```
# Todos os testes
dbt test --profiles-dir .dbt

# Só um modelo
dbt test --profiles-dir .dbt --select dim_player

# Só um tipo de teste
dbt test --profiles-dir .dbt --select test_type:unique
```

■ Pergunta de entrevista: Como você garante qualidade de dados em produção?

Tenho 26 testes declarativos no dbt cobrindo unique, not_null, accepted_values e relationships. Eles rodam automaticamente no CI (GitHub Actions) a cada PR — qualquer modelo quebrado bloqueia o merge. Em produção, o Dagster re-executa os testes após cada materialização. Para escala maior adicionaria Elementary para rastrear anomalias históricas (ex: volume de linhas caiu 20% sem motivo aparente) e testes customizados em SQL para regras de negócio mais específicas.

PARTE 8 — Guia de Entrevista Pleno/Sênior

8.1 O pitch de 2 minutos

"Construí um pipeline de dados end-to-end para análise de estatísticas da NBA. O dado vem do Basketball Reference via scraping com Selenium — o site bloqueia requests diretos. Os CSVs são carregados no PostgreSQL via dbt seed e transformados em 3 camadas: staging para limpeza, intermediate para regras de negócio como de-duplicação de jogadores trocados, e marts com Star Schema. O ambiente roda em Docker Compose, a orquestração é feita com Dagster (assets nativos dbt, schedule semanal), e o CI/CD no GitHub Actions valida os 26 testes de qualidade em cada PR. A documentação é publicada automaticamente no GitHub Pages via dbt docs."

8.2 Perguntas — nível Pleno

■ Pergunta de entrevista: Qual a diferença entre ETL e ELT?

ETL transforma antes de carregar — necessário quando storage era caro. ELT carrega bruto e transforma dentro do banco — padrão moderno. dbt é fundamentalmente ELT: os dados chegam via seed sem transformação, o banco executa os SELECTs dos modelos. Vantagem: dados brutos sempre disponíveis para análises não previstas sem re-extrair da fonte.

■ Pergunta de entrevista: O que é materialização no dbt e quando usar cada tipo?

VIEW: sem storage, executa na consulta — ideal para staging e intermediate. TABLE: persiste fisicamente — ideal para marts consultados por analistas. INCREMENTAL: processa só registros novos — para tabelas grandes em produção. EPHEMERAL: CTE inline, nem cria objeto no banco — para lógica auxiliar simples. No projeto: staging e intermediate são VIEW, marts são TABLE.

■ Pergunta de entrevista: O que é um DAG e como o dbt usa esse conceito?

DAG (Directed Acyclic Graph) é um grafo de dependências sem ciclos. No dbt, cada ref() em um modelo cria uma aresta no grafo. O dbt calcula automaticamente a ordem de execução e paraleliza modelos sem dependência mútua. fct_player_season_stats só roda depois de dim_player e dim_team existirem, que só rodam depois dos intermediates, que dependem do staging. Nunca precisei escrever essa ordem manualmente.

8.3 Perguntas — nível Sênior

■ Pergunta de entrevista: Como você escalaria esse pipeline para múltiplas temporadas?

Três mudanças principais: (1) coluna season em todos os seeds e modelos para identificar a temporada de cada registro; (2) materialização INCREMENTAL na fct — processa só os dados da temporada atual sem reconstruir histórico; (3) SCD Type 2 na dim_player para rastrear mudanças de time entre temporadas com effective_from/effective_to. O Dagster gerenciaria qual partição de dados está desatualizada e re-materializaria só ela.

■ Pergunta de entrevista: Como o CI garante que nenhum modelo quebrado chega em produção?

O GitHub Actions ci.yml roda em cada PR: sobe PostgreSQL como service container, instala dbt, escreve o profiles.yml com as credenciais de CI, e executa compile → seed → run → test em sequência. Se qualquer

step falhar, o PR fica bloqueado. Em projetos maiores usaria dbt slim CI (--state flag) para rodar apenas modelos que mudaram no PR, reduzindo tempo de 10 minutos para 1-2 minutos.

■ Pergunta de entrevista: Por que Dagster e não Airflow para orquestrar o pipeline dbt?

Para este caso específico: @dbt_assets importa todos os 8 modelos dbt automaticamente a partir do manifest.json — sem escrever uma task por modelo. No Airflow eu escreveria BashOperators manuais para seed, run e test, sem granularidade por modelo. Além disso, a UI do Dagster mostra o grafo de assets com status de saúde por modelo, não apenas 'o DAG passou ou falhou'. Em ambiente com Airflow já estabelecido usaria o provider astronomer-cosmos que traz integração similar.

8.4 Red flags para evitar

■ Não diga	■ Diga no lugar
'Segui um tutorial'	'Enfrentei o problema X e resolvi assim...'
'dbt é tipo um ORM'	'dbt é transformador SQL com gestão de DAG e testes'
'Não sei por que funciona'	'A razão dessa escolha foi... (explique trade-offs)'
'É só um projeto pessoal simples'	'Implementei padrões de produção: CI, Docker, Dagster'
'Nunca usei em produção'	'Apliquei as práticas que usaria em prod: env_var, CI, docs'

PARTE 9 — Próximos Passos e Glossário

9.1 Roadmap de evolução

Evolução	Dificuldade	O que demonstra
Modelos incrementais + múltiplas temporadas	■■■	Modelagem temporal, eficiência em escala
SCD Type 2 em dim_player	■■■	Histórico de mudanças em dimensões
Elementary (observabilidade de qualidade)	■■	Monitoramento contínuo de anomalias
Dashboard com Metabase ou Superset	■■	Entrega de valor visível ao usuário final
Testes customizados em SQL (dbt macros)	■■■	Qualidade de dados avançada
Migrar para Snowflake ou BigQuery	■■	Cloud data warehouse de mercado
dbt Snapshots para histórico de contratos	■■	Rastreamento de mudanças ao longo do tempo

9.2 Glossário completo

Termo	Definição
Asset (Dagster)	Dado que deve existir — o conceito central do Dagster (equivalente a tabela/view)
CI/CD	Continuous Integration/Delivery — validação e publicação automatizadas a cada mudança
DAG	Directed Acyclic Graph — grafo de dependências sem ciclos (base do dbt e Dagster)
ELT	Extract-Load-Transform — dados brutos carregados primeiro, transformados no banco
Grain	A granularidade da tabela fato — o que cada linha representa
Jinja2	Sistema de templates Python usado pelo dbt para lógica em SQL <code>{{ ref() }}</code> , <code>{% if %}</code>
Lineage	Mapa de dependências entre modelos — mostra de onde os dados vêm e para onde vão
Macro	Função Jinja reutilizável no dbt (ex: <code>generate_surrogate_key</code>)
Materialização	Como o dbt persiste um modelo: view, table, incremental ou ephemeral
Profile	Arquivo de configuração com credenciais de conexão (<code>.dbt/profiles.yml</code>)
ref()	Função dbt que cria dependência entre modelos e resolve o nome completo da tabela
SCD Type 2	Slowly Changing Dimension — técnica para rastrear histórico de mudanças em dimensões
Schema	Namespace no PostgreSQL que agrupa tabelas (<code>analytics_raw</code> , <code>analytics_marts...</code>)
Seed	CSV carregado no banco pelo dbt seed — usado para dados de referência ou ingestão
Star Schema	Modelo dimensional com tabela fato central conectada a tabelas dimensão
Surrogate Key	ID artificial gerado (MD5, UUID) — estável e independente dos dados de negócio